Chapter 1

Optimization

In Exercise 13.3 in the previous chapter you were asked to find the best launch angle for a human cannonball, meaning the angle that maximizes the distance traveled before landing. This kind of problem, finding minimums and maximums, is called *optimization*.

In this chapter, we'll solve a similar problem, finding the best launch angle for a baseball. We'll solve the problem two ways, first running simulations with a range of values and plotting the results, then using a MATLAB function that automates the process, `fminsearch`.

1.1 Optimal Baseball

In the previous chapter we wrote functions to simulate the flight of a baseball with a known initial velocity. Now we'll use that code to find the launch angle that maximizes *range*, that is, the distance the ball travels before landing.

First, we need an event function to stop the simulation when the ball lands.

```
function [value, isterminal, direction] = event_landing(t, xx)
    % Event function for ode solver to trigger when vertical position
    % is zero
    value = xx(3);
    isterminal = 1;
    direction = -1;
end
```

This is similar to the event function we saw in Chapter 12.3, except that it uses `xx(3)` as the event value, which is the element of the state vector corresponding to the y-position. This event function stops the simulation when the altitude of the ball is 0 and falling.

Now we can call `ode45` like this:

```
% Initial conditions
xx_init = [0 30 1 40]';
```

```
% Solve with ode45
tspan = [0 10];
options = odeset('Events', @event_landing);
[tout, Xout] = ode45(@state_transition, tspan, xx_init, options);
```

The initial position of the ball is 1 m above home plate. The ball's initial velocity is 40 m/s in the x-direction and 30 m/s in the y-direction.

The maximum duration of the simulation is 10 s, but we expect an event to stop the simulation first. We can get the final values of the simulation like this:

```
>> tout(end)
ans =
    5.9307
>> Xout(end,:)
ans =
  112.2167   11.9139   -0.0000  -36.2661
```

The final time is 5.9 s. The final x-position is 112 m; the final y-position is 0 m, as expected.

As we saw in Chapter 13.3, we can plot the *trajectory* the baseball from launch, on the left, to landing, on the right. This plot is repeated in Figure 1.1

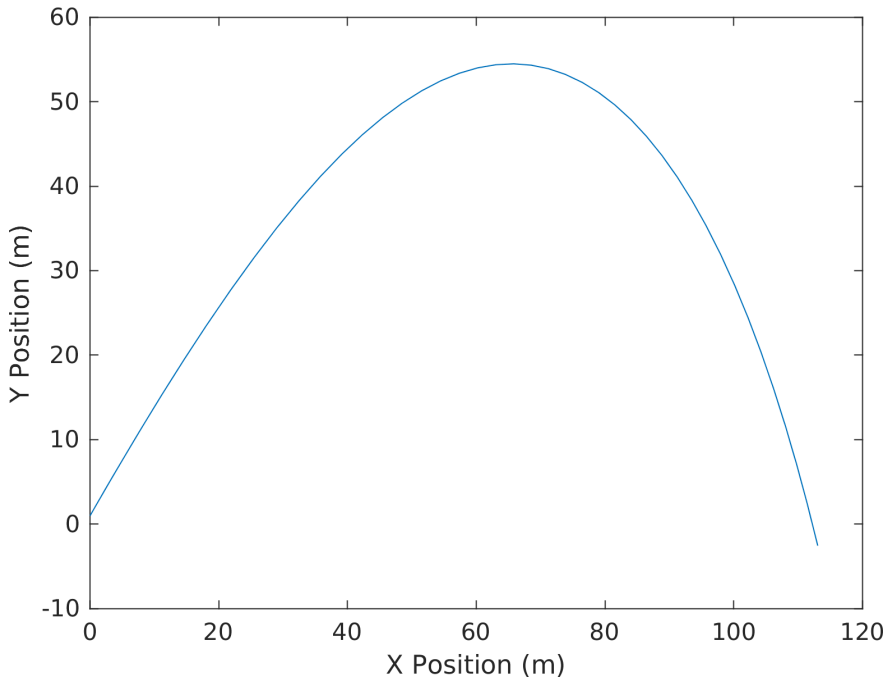


Figure 1.1: Simulated flight of a baseball plotted as a trajectory

1.2 Exhaustive Search: Range Versus Angle

Now we'll simulate the trajectory of the baseball with a range of launch angles. First, we'll take the code we have and wrap it in a function that takes the launch angle as an input

variable, runs the simulation, and returns the distance the ball travels (Listing 1.1).

Listing 1.1: A function that takes the launch angle of a baseball and returns the distance it travels

```
function res = baseball_range(theta)
    P = [0; 1];
    v = 50;
    [vx, vy] = pol2cart(theta, v);

    V = [vx; vy];      % initial velocity in m/s
    W = [P; V];        % initial condition

    tspan = [0 10];
    options = odeset('Events', @event_func);
    [T, M] = ode45(@rate_func, tspan, W, options);

    res = M(end, 1);
end
```

The launch angle, `theta`, is in radians. The magnitude of velocity, `v`, is always 50 m/s. We use `pol2cart` to convert the angle and magnitude (*polar* coordinates) to Cartesian components, `vx` and `vy`.

After running the simulation we extract the final x-position and return it as an output variable.

We can run this function for a range of angles like this:

```
    thetas = linspace(0, pi/2);
    for i = 1:length(thetas)
        ranges(i) = baseball_range(thetas(i));
    end
```

And then plot `ranges` as a function of `thetas`:

```
    plot(thetas, ranges)
```

Figure 1.2 shows the result. As expected, the ball does not travel far if it's hit nearly horizontal or vertical. The peak is apparently near 0.7 rad. This is an example of the most basic optimization technique, *exhaustive search*. In this case, it is computationally feasible to generate (almost) the entire solutions space, the full set of possible launch angles, and then simply select the best one.

Considering that our model is only approximate, this result might be good enough. But if we want to find the peak more precisely, we can use `fminsearch`.

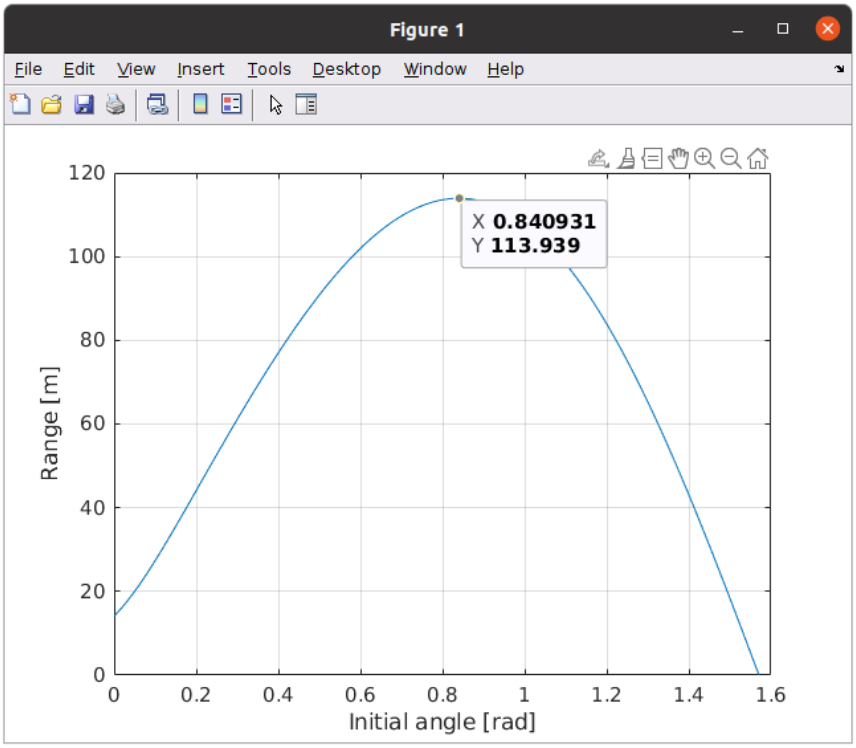


Figure 1.2: Exhaustive search for maximum range using the *Data tips* graphical tool to choose the near-optimal solution

1.3 fminsearch

The `fminsearch` function is similar to `fzero`, which we saw in Chapter 8. Recall that `fzero` takes a function handle and an initial guess, and it returns a root of the function. The `fminsearch` function uses this same interface to implement a nonlinear optimization to search for a value that minimizes the user-supplied function.

If we want to find the maximum of a function, rather than the minimum, we can still use `fminsearch` by writing a short function that negates the function we want to maximize. In the baseball example, the function we want to maximize is `baseball_range`; we can wrap it in another function like this:

```
function nrange = negative_range(theta)
% Return -1*range for use with function minimization
nrange = -1.0*baseball_range(theta);
end
```

And then we call `fminsearch` like this:

```
    % Inital estimate of solution
    theta0 = 0;
    % Optimization
    [x, fval] = fminsearch(@negative_range, theta0)
```

which would return

```
    x =
    0.8403
fval =
 -113.9392
```

The optimal launch angle for the baseball is 0.84 rad; launched at that angle, the ball travels almost 114 m.

If you're curious about how `fminsearch` works, see "How fminsearch Works" on page 8.

1.4 Logical Errors

The `fminsearch` algorithm often does a great job of finding a local minimum for a wide range of non-linear function, but there are lots of ways that things can go wrong. This is a common issue with black-box solutions where the process of finding the solution is not clear. The best way to avoid these logical bugs—where the program is doing exactly what we told it, but still doesn't give the result we intended—is to peer into the black-box. We can do this by setting some options available in the `fminsearch` interface and looking at more of the outputs. Here is an example:

```
    % Optimization options
    options = optimset('Display','iter', ...
        'PlotFcns', {@optimplotx, @optimplotfval, @optimplotfunccount});
    % Call optimization
    [x, fval, exitflag, output] = fminsearch(@negative_range, theta0, options)
```

In the snippet above we use the `optimset` function to set parameters to values. This is a common idiom in MATLAB where parameters are specified by *Name* and *Value*. Here we are setting the parameter named `Dispay` to the string value `'iter'`. Then we set the parameter named `PlotFcns` to a cell array value with three function handles. The eclipses, `...`, tell MATLAB to continue the same command on the next line. We have also added two additional output variables so that the algorithm is more verbose, giving us both the solution and some information about the solution.

And here is what we get in return. The `Dispay='iter'` option results in the following output to the Command Window

```
    Iteration   Func-count     min f(x)         Procedure
     0            1         -14.0976
     1            2         -14.1229         initial simplex
     2            4         -14.1736         expand
     3            6         -14.2755         expand
     4            8         -14.4816         expand
     5           10         -14.9023         expand
     6           12         -15.7779         expand
```

```
     7           14          -17.663         expand
     8           16         -21.9205         expand
     9           18         -31.8432         expand
    10           20         -53.6809         expand
    11           22         -92.3186         expand
    12           24         -112.788         reflect
    13           26         -113.243         contract outside
    14           28         -113.923         contract inside
    15           30         -113.923         contract inside
    16           32         -113.927         contract inside
    17           34         -113.939         contract inside
    18           36         -113.939         contract inside
    19           38         -113.939         contract inside
    20           40         -113.939         contract inside
    21           42         -113.939         contract inside
    22           44         -113.939         contract inside
    23           46         -113.939         contract inside
    24           48         -113.939         contract inside

Optimization terminated:
 the current x satisfies the termination criteria using OPTIONS.TolX of 1.000000e-04
 and F(X) satisfies the convergence criteria using OPTIONS.TolFun of 1.000000e-04
```

This gives us some diagnostic information about each step in the optimization and tells us that the solution converged, which is a good sign.

Setting the `PlotFcns` option to the three built-in plotting functions, results in a new figure window as shown in Figure 1.3. If you are quick, you might notice that this plot is generated incrementally as the optimization iterates. This can be helpful for more computationally demanding scenarios that might take minutes or hours. Having some incremental status information helps.

The two additional output variables also help us make sure the answer we received is what we intended.

```
exitflag =
     1
output =
  struct with fields:

    iterations: 24
     funcCount: 48
     algorithm: 'Nelder-Mead simplex direct search'
       message: 'Optimization terminated: the current x satisfies the
       termination criteria using OPTIONS.TolX of 1.000000e-04
        and F(X) satisfies the convergence criteria using
        OPTIONS.TolFun of 1.000000e-04 '
```

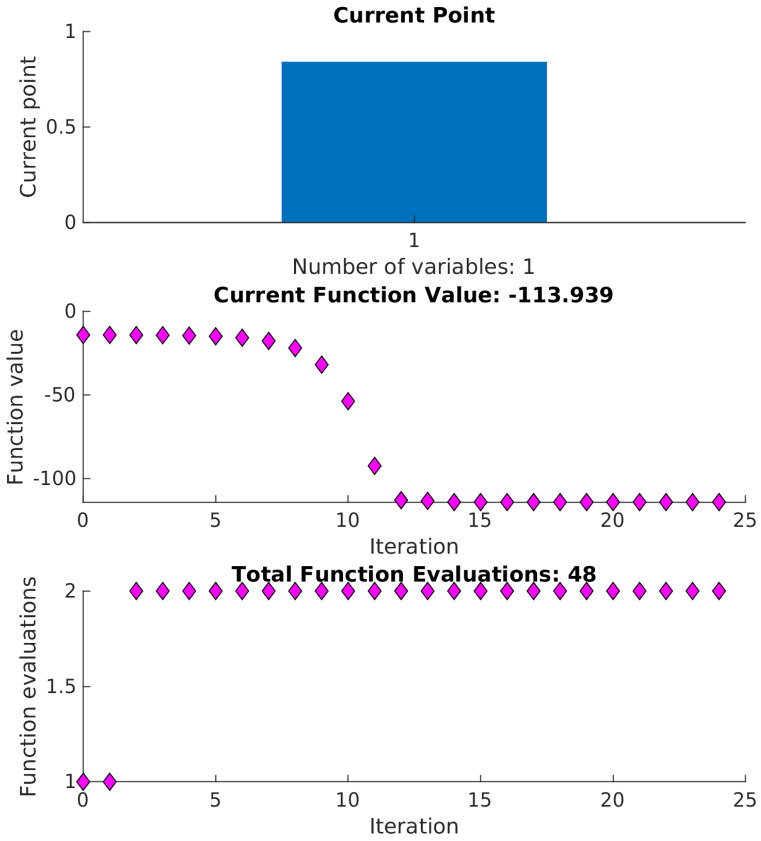


Figure 1.3: Figure resulting from setting
`'PlotFcns' = @optimplotx, @optimplotfval, @optimplotfunccount`

The `exitflag` will be `1` when the solution converged, `0` when it exceeds the allowed number of iterations and `-1` if the search ended do to a user-defined exit criteria—similar to the event function we used with `ode45`. The `output` variable is a structure with fields to characterize the process.

1.5 How fminsearch Works

According to the MATLAB documentation, `fminsearch` uses the Nelder-Mead simplex algorithm. You can read about it at `https://greenteapress.com/matlab/nelder`, but you might find it overwhelming.

To give you a sense of how it works, I will present a simpler algorithm, the *golden-section search*. Suppose we're trying to find the minimum of a function of a single variable, $f(x)$.

As a starting place, assume that we have evaluated the function at three places, x_1, x_2, and x_3, and found that x_2 yields the lowest value. Figure 1.4 shows this initial state.

We will assume that $f(x)$ is continuous and *unimodal* in this range, which means that there is exactly one minimum between x_1 and x_3.

The next step is to choose a fourth point, x_4, and evaluate $f(x_4)$. There are two possible outcomes, depending on whether $f(x_4)$ is greater than $f(x_2)$ or not. Figure 1.5 shows the two possible states.

If $f(x_4)$ is less than $f(x_2)$ (shown on the left), the minimum must be between x_2 and x_3, so we would discard x_1 and proceed with the new triple (x_2, x_4, x_3).

If $f(x_4)$ is greater than $f(x_2)$ (shown on the right), the local minimum must be between x_1 and x_4, so we would discard x_3 and proceed with the new triple (x_1, x_2, x_4).

Either way, the range gets smaller and our estimate of the optimal value of x gets better.

This method works for almost any value of x_4, but some choices are better than others. You might be tempted to bisect the interval between x_2 and x_3, but that turns out not to be the best choice. You can read about a better option at `https://greenteapress.com/matlab/golden`.

1.6 Animation

Animation is a useful tool for checking the results of a physical model. If something is wrong, animation can make it obvious. There are two ways to do animation in MATLAB. One is to use `getframe` to capture a series of images and then use `movie` to play them back. This method is a bit involved, but allows you to generate a standalone video file, for example, a MP4 file.

The more informal way is to draw a series of plots. You may have seen this when MATLAB incrementally generated the plots in Figure 1.3. Listing 1.2 is a snippet that animates the results of a baseball simulation:

Listing 1.2: A snippet that animates the results of a baseball simulation

```
% Animate
% Solve with ode45
tspan = [0 10];
xx_init = [0 30 1 40];
```

```
    options = odeset('Events', @event_landing);
    [tout, Xout] = ode45(@state_transition, tspan, xx_init, options);

    % Setup figure
    figure(4);
    clf();
    x = Xout(:,1);
    y = Xout(:,3);
    minmax = [min([x]), max([x]), min([y]), max([y])];
    % Plot each element in the solution
    for ii=1:round(length(tout)*4/5)
        clf;
        hold on;
        plot(x(1:ii), y(1:ii), 'b:')
        plot(x(ii), y(ii), 'ro')
        axis(minmax)
        xlabel('x position [m]')
        ylabel('y position [m]')
        drawnow;
        if ii < length(tout)
            dt = tout(ii+1) - tout(ii);
            pause(dt);
        end
    end
```

A vector of four elements, `minmax` is used inside the loop to set the axes of the figure. This is necessary because otherwise MATLAB scales the figure each time through the loop, so the axes keep changing, which makes the animation hard to watch.

Each time through the loop, `animate` uses `clf` to clear the figure and `axis` to reset the axes. Then it plots a circle to represent the position of the baseball.

We have to call `drawnow` after `plot` so that MATLAB actually displays each plot. Otherwise, it waits until you finish drawing all the figures and then updates the display.

One limitation of this kind of animation is that the speed of the animation depends on how fast your computer can generate the plots. Since the results from `ode45` are usually not equally spaced in time, your animation might slow down where `ode45` takes small time steps and speed up where the time steps are larger.

One way to fix this problem is to change the way we specify `tspan`. Here's an example:

```
    tspan = 0:0.1:10;
```

The result is a vector that goes from 0 to 10 with a step size of 0.1. Passing `tspan` to `ode45` in this form doesn't affect the accuracy of the results; `ode45` still uses variable time steps to generate the estimates, but then it interpolates them before returning the results.

With equal time steps, the animation should be smoother.

Another option is to use `pause` to play the animation in real time. After drawing each frame and calling `drawnow`, you can compute the time until the next frame and use `pause` to wait:

```
dt = tout(ii+1) - tout(ii);
pause(dt);
```

A limitation of this method is that it ignores the time required to draw the figure, so it tends to run slow, especially if the figure is complex or the time step is small.

1.7 Chapter Review

This chapter presented two new tools, `fminsearch` and `animate`. The MATLAB function `fminsearch` searches efficiently for the minimum of a function and can be adapted to search for the maximum, too. The `animate` function is one I wrote to read results from `ode45` and generate an animation; the version in this chapter works with the results from the baseball simulation, but it can be adapted for other simulations.

In the exercises below, you have a chance to extend the example from this chapter and bring together many of the tools we have used so far.

In the next chapter, we move on to a new example, celestial mechanics, which describes the motion of planets and other bodies in outer space.

1.8 Exercises

Before you go on, you might want to work on the following exercises.

Exercise 1.1. Manny Ramirez is a former member of the Boston Red Sox who was famous for his relaxed attitude. The goal of this exercise is to solve the following Manny-inspired problem:

> What is the minimum effort required to hit a home run in Fenway Park?

Fenway Park is a baseball stadium in Boston, Massachusetts. One of its most famous features is the "Green Monster," which is a wall in left field that is unusually close to home plate, only 310 feet away. To compensate for the short distance, the wall is unusually high, at 37 feet.

You can solve this problem in two steps:

1. For a given velocity, find the launch angle that maximizes the height of the ball when it reaches the wall. Notice that this is not quite the same as the angle that maximizes the distance the ball travels.

2. Find the minimal velocity that clears the wall, given that it has the optimal launch angle. Hint: this is actually a root-finding problem, not an optimization problem.

Exercise 1.2. A golf ball hit with backspin generates lift, which might increase the distance it travels, but the energy that goes into generating spin probably comes at the cost of lower initial velocity.

Write a simulation of the flight of a golf ball and use it to find the launch angle and allocation of spin and initial velocity (for a fixed energy budget) that maximizes the horizontal range of the ball in the air.

The lift of a spinning ball is due to the Magnus force (see `https://greenteapress.com/matlab/magnus`), which is perpendicular to the axis of spin and the path of flight. The coefficient of lift is proportional to the spin rate; for a ball spinning at 3000 rpm it is about 0.1. The coefficient of drag of a golf ball is about 0.2 as long as the ball is moving faster than 20 m/s.

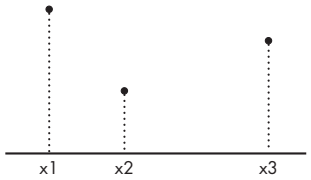


Figure 1.4: Initial state of a golden-section search

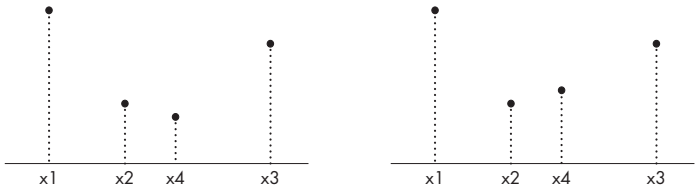


Figure 1.5: Possible states of a golden-section search after evaluating $f(x_4)$